

$3.93M a month left in shopping carts

Cart leakage and retention · Databricks SQL · Parth Dhawan · October 2026

2.4 GB · 20.7M shopping events · 4.52M carts · 465 categories · Oct 2019 to Feb 2020

Three out of four carts are never bought

76.5%

of carts abandoned

3.46M of 4.52M carts were not bought within 7 days, and the rate holds at 75% to 78% every month.

$3.93M

left unbought a month

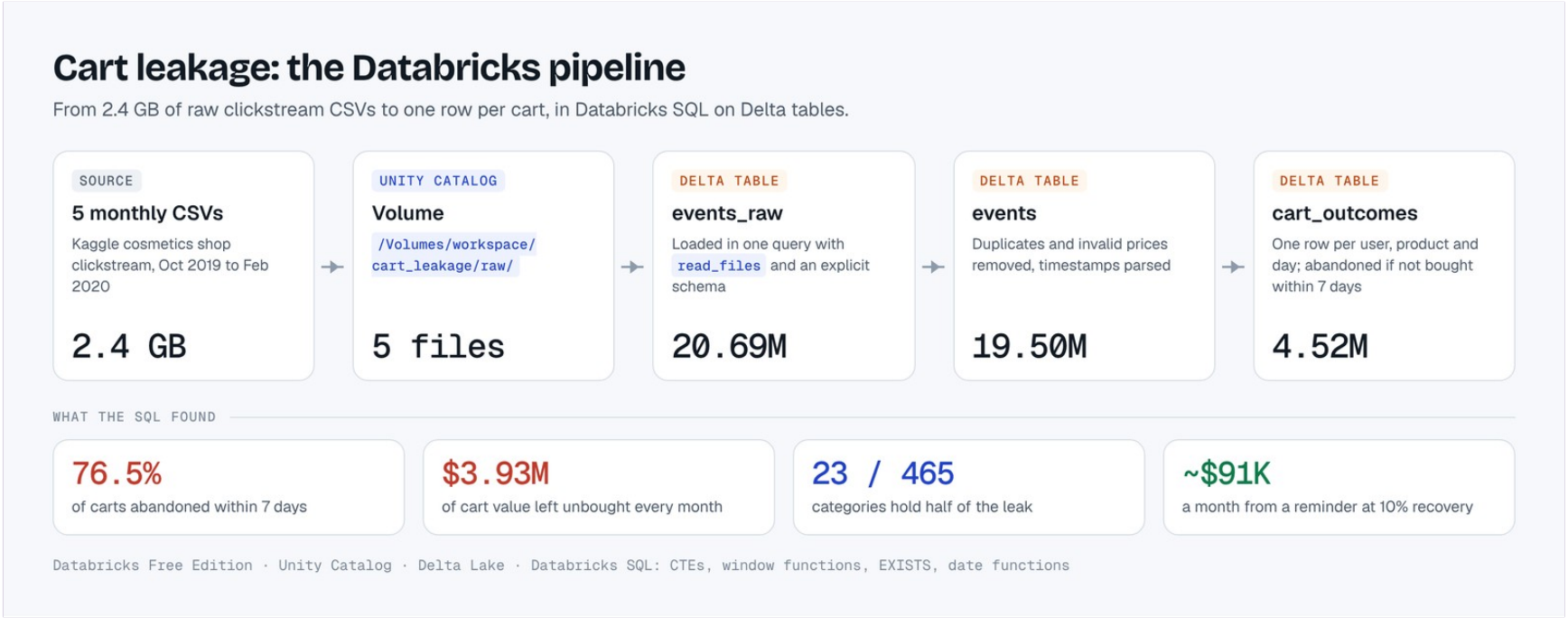
Half of it sits in just 23 of 465 categories, so the fix can start small.

~$91K

a month from a reminder

If a reminder wins back 10% of carts, after a 25% margin and $0.01 per message.

2.4 GB of raw clicks to one row per cart, in Databricks



Each step is its own Delta table, so every number can be traced back to the raw clicks.

Cleaned before counting

A1_queries_as_run.sql — Databricks SQL · catalog workspace · schema cart_leakage

```sql
-- 1. Load all 5 monthly CSVs from the volume
CREATE OR REPLACE TABLE workspace.cart_leakage.events_raw AS
SELECT * FROM read_files('/Volumes/workspace/cart_leakage/raw/',
  format => 'csv', header => true, pathGlobFilter => '*.csv',
  schema => 'event_time STRING, event_type STRING, product_id BIGINT, category_id BIGINT, category_code STRING, brand STRING, price
DOUBLE, user_id BIGINT, user_session STRING');

-- 4. Clean events table
CREATE OR REPLACE TABLE workspace.cart_leakage.events AS
SELECT DISTINCT to_timestamp(replace(event_time, ' UTC', '')) AS event_ts, event_type, product_id, category_id,
  COALESCE(category_code, concat('cat_', category_id)) AS category, COALESCE(brand, 'unknown') AS brand,
  price, user_id, user_session
FROM workspace.cart_leakage.events_raw WHERE price IS NOT NULL AND price > 0;
```

events_raw **20.69M rows** events (cleaned) **19.50M rows** removed **5.8%**

19.50M events

kept of 20.69M: duplicates and invalid prices removed (5.8%)

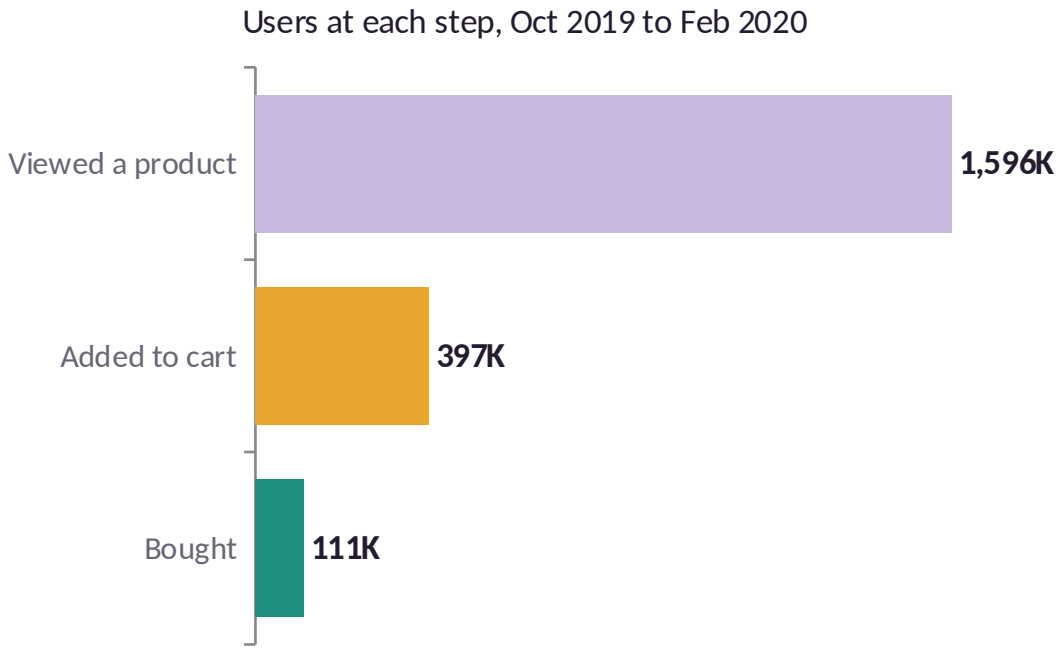
4.52M carts

one per user, product and day, so repeat clicks count once

7-day window

abandoned = the same product not bought within 7 days

The leak is at the cart, not the shelf



24.9%

of viewers add something to a cart

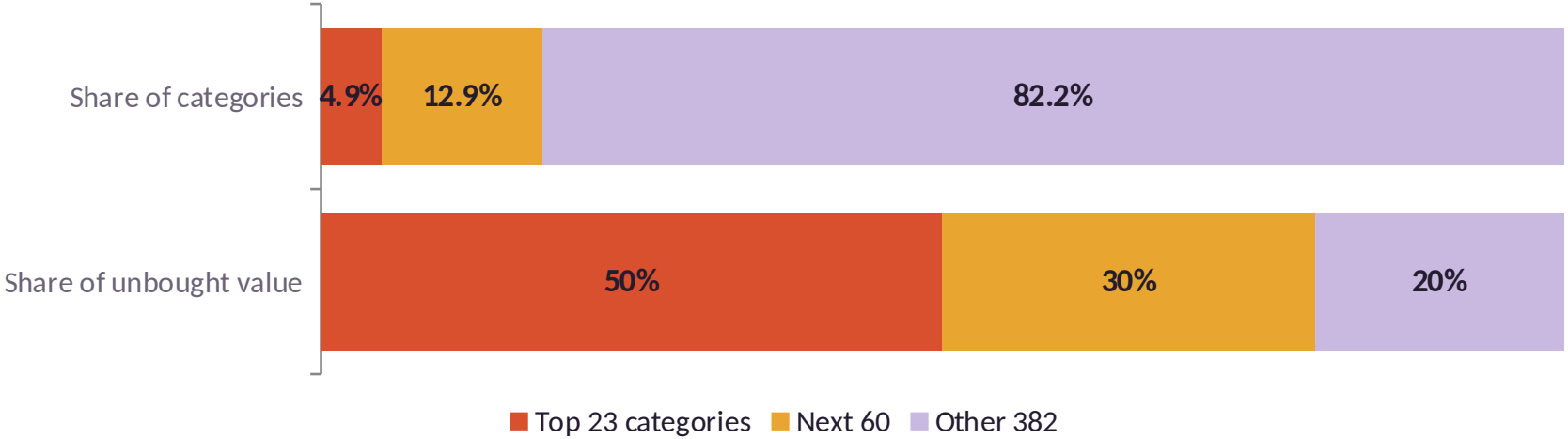
27.8%

of those who add to a cart go on to buy

84%

of sessions with a cart end with no purchase

Half the leak sits in 23 of 465 categories



5% → 50%

23 categories, 5% of the range, hold half of all unbought value

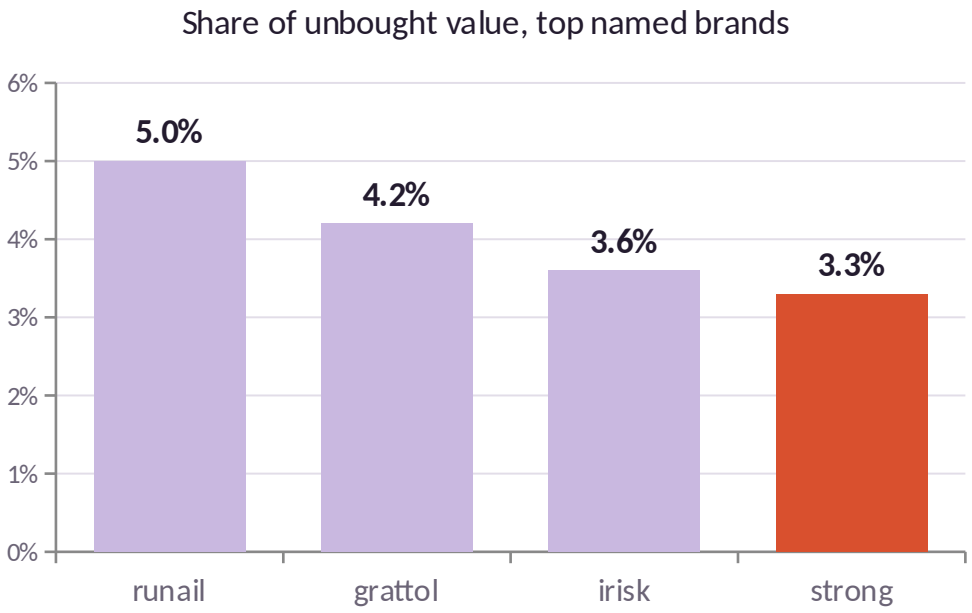
18% → 80%

83 categories hold 80% of it

7.3%

the most any single category holds, so there is no one culprit

Rank by value, not by cart count



strong

Only 4,036 carts, yet 3.3% of the leak: about $189 per abandoned cart. A few expensive carts matter more than many cheap ones.

40.1% of the leak has no brand recorded, so brand results cover the rest.

New buyers rarely come back



9.2%

of November and December first-time buyers bought again the next month.

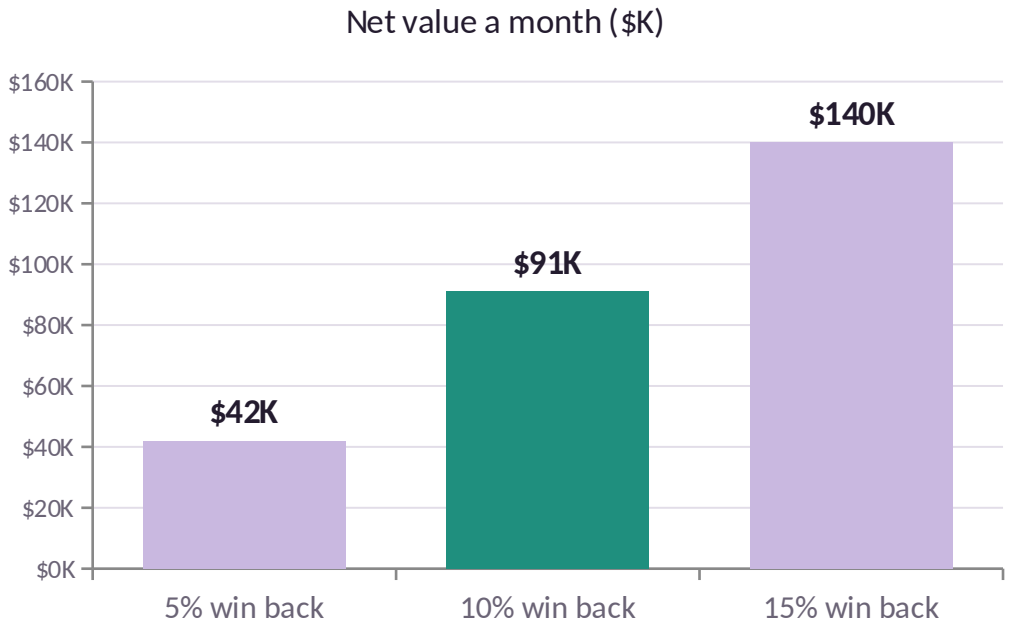
A reminder brings back carts; a second-order offer could bring back customers.

What I caught

October showed 18.5%, twice the rate.

The data starts in October, so long-time customers look new that month. I reported it as unreliable instead of as growth.

A reminder is worth about $91K a month at 10%



How it is priced

- Recovered sales × 25% margin, minus $0.01 per reminder to every abandoned cart.
- At 10%: $393K of sales recovered a month.
- Margin and cost are my assumptions; the win-back rate is untested.

Next steps: test, then scale

1 Start the reminder with the 23 categories that hold half of the unbought value.

2 Run it against a random holdout group, so the real win-back rate replaces my 10% assumption.

3 Add a second-order offer for new buyers: only 9.2% come back on their own.

4 Fix the data at the source: 42% of events have no brand and 98% no category name.

Happy to walk you through the SQL

Every query is in the repo exactly as it ran in Databricks.

github.com/parthdhawan921-ops/cart-leakage-analysis

parthdhawan921-ops.github.io/Parth_Personal/#cart

parthdhawan921@gmail.com · +91 70823 35959